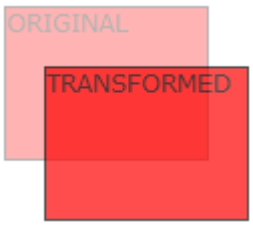
CSS3 Transformations, Transitions & Animations

CSS3 2D Transforms

In this chapter you will learn about the following 2D transformation methods:

- `translate()`
- `rotate()`
- `scale()`
- `skewX()`
- `skewY()`
- `matrix()`

The translate() Method



The `translate()` method moves an element from its current position (according to the parameters given for the X-axis and the Y-axis).

The following example moves the element 50 pixels to the right, and 100 pixels down from its current position:

```
div {
    -ms-transform: translate(50px, 100px); /* IE 9 */
    -webkit-transform: translate(50px, 100px); /* Safari */
    transform: translate(50px, 100px);
}
```

The rotate() Method



The `rotate()` method rotates an element clockwise or counter-clockwise according to a given degree.

The following example rotates the element clockwise with 20 degrees:

```
div {
    -ms-transform: rotate(20deg); /* IE 9 */
    -webkit-transform: rotate(20deg); /* Safari */
    transform: rotate(20deg);
}
```

Using negative values will rotate the element counter-clockwise.

The following example rotates the element counter-clockwise with 20 degrees:

```
div {
    -ms-transform: rotate(-20deg); /* IE 9 */
    -webkit-transform: rotate(-20deg); /* Safari */
    transform: rotate(-20deg);
}
```

The scale() Method



The `scale()` method increases or decreases the size of an element (according to the parameters given for the width and height).

The following example increases the element to be two times of its original width, and three times of its original height:

```
div {
    -ms-transform: scale(2, 3); /* IE 9 */
    -webkit-transform: scale(2, 3); /* Safari */
    transform: scale(2, 3);
}
```

The following example decreases the element to be half of its original width and height:

```
div {
    -ms-transform: scale(0.5, 0.5); /* IE 9 */
```

```
    -webkit-transform: scale(0.5, 0.5); /* Safari */
    transform: scale(0.5, 0.5);
}
```

The skewX() Method

The `skewX()` method skews an element along the X-axis by the given angle.

The following example skews the element 20 degrees along the X-axis:

```
div {
    -ms-transform: skewX(20deg); /* IE 9 */
    -webkit-transform: skewX(20deg); /* Safari */
    transform: skewX(20deg);
}
```

The skewY() Method

The `skewY()` method skews an element along the Y-axis by the given angle.

The following example skews the element 20 degrees along the Y-axis:

```
div {
    -ms-transform: skewY(20deg); /* IE 9 */
    -webkit-transform: skewY(20deg); /* Safari */
    transform: skewY(20deg);
}
```

The skew() Method

The `skew()` method skews an element along the X and Y-axis by the given angles.

The following example skews the element 20 degrees along the X-axis, and 10 degrees along the Y-axis:

```
div {
    -ms-transform: skew(20deg, 10deg); /* IE 9 */
    -webkit-transform: skew(20deg, 10deg); /* Safari */
    transform: skew(20deg, 10deg);
}
```

If the second parameter is not specified, it has a zero value. So, the following example skews the element 20 degrees along the X-axis:

```
div {
    -ms-transform: skew(20deg); /* IE 9 */
    -webkit-transform: skew(20deg); /* Safari */
    transform: skew(20deg);
}
```

The matrix() Method



The `matrix()` method combines all the 2D transform methods into one.

The matrix() method take six parameters, containing mathematic functions, which allows you to rotate, scale, move (translate), and skew elements.

The parameters are as follow:
matrix(scaleX(),skewY(),skewX(),scaleY(),translateX(),translateY()):

```
div {
    -ms-transform: matrix(1, -0.3, 0, 1, 0, 0); /* IE 9 */
    -webkit-transform: matrix(1, -0.3, 0, 1, 0, 0); /* Safari */
    transform: matrix(1, -0.3, 0, 1, 0, 0);
}
```

CSS3 3D Transforms

In this chapter you will learn about the following 3D transformation methods:

- `rotateX()`
- `rotateY()`
- `rotateZ()`

The rotateX() Method



The `rotateX()` method rotates an element around its X-axis at a given degree:

```
div {
    -webkit-transform: rotateX(150deg); /* Safari */
    transform: rotateX(150deg);
}
```

The rotateY() Method



The `rotateY()` method rotates an element around its Y-axis at a given degree:

```
div {
    -webkit-transform: rotateY(130deg); /* Safari */
    transform: rotateY(130deg);
}
```

The rotateZ() Method

The `rotateZ()` method rotates an element around its Z-axis at a given degree:

```
div {
    -webkit-transform: rotateZ(90deg); /* Safari */
    transform: rotateZ(90deg);
}
```

CSS3 Transitions

CSS3 transitions allows you to change property values smoothly (from one value to another), over a given duration.

How to Use CSS3 Transitions?

To create a transition effect, you must specify two things:

- the CSS property you want to add an effect to
- the duration of the effect

Note: If the duration part is not specified, the transition will have no effect, because the default value is 0.

The following example shows a 100px * 100px red element. The element has also specified a transition effect for the width property, with a duration of 2 seconds:

```
div {
    width: 100px;
    height: 100px;
    background: red;
    -webkit-transition: width 2s; /* Safari */
    transition: width 2s;
}
```

The transition effect will start when the specified CSS property (width) changes value.

Now, let us specify a new value for the width property when a user mouses over the element:

```
div:hover {
    width: 300px;
}
```

Notice that when the cursor mouses out of the element, it will gradually change back to its original style.

Change Several Property Values

The following example adds a transition effect for both the width and height property, with a duration of 2 seconds for the width and 4 seconds for the height:

```
div {
    -webkit-transition: width 2s, height 4s; /* Safari */
    transition: width 2s, height 4s;
}
```

Specify the Speed Curve of the Transition

The `transition-timing-function` property specifies the speed curve of the transition effect.

The transition-timing-function property can have the following values:

- `ease` - specifies a transition effect with a slow start, then fast, then end slowly (this is default)
- `linear` - specifies a transition effect with the same speed from start to end
- `ease-in` - specifies a transition effect with a slow start
- `ease-out` - specifies a transition effect with a slow end
- `ease-in-out` - specifies a transition effect with a slow start and end
- `cubic-bezier(n,n,n,n)` - lets you define your own values in a cubic-bezier function

The following example shows the some of the different speed curves that can be used:

```
div {
    width: 100px;
    height: 100px;
    background: red;
    -webkit-transition: width 2s; /* Safari */
    transition: width 2s;
}

/* Standard syntax */
#div1 {transition-timing-function: linear;}
#div2 {transition-timing-function: ease;}
#div3 {transition-timing-function: ease-in;}
#div4 {transition-timing-function: ease-out;}
#div5 {transition-timing-function: ease-in-out;}

div:hover {
    width: 300px;
}
```

Delay the Transition Effect

The `transition-delay` property specifies a delay (in seconds) for the transition effect.

The following example has a 1 second delay before starting:

```
div {
    -webkit-transition-delay: 1s; /* Safari */
    transition-delay: 1s;
}
```

Transition + Transformation

The following example also adds a transformation to the transition effect:

```
div {
    width: 100px;
    height: 100px;
    background: red;
    -webkit-transition: width 2s, height 2s, -webkit-transform 2s; /* Safari */
    transition: width 2s, height 2s, transform 2s;
}

div:hover {
    width: 300px;
    height: 300px;
    -webkit-transform: rotate(180deg); /* Safari */
    transform: rotate(180deg);
}
```